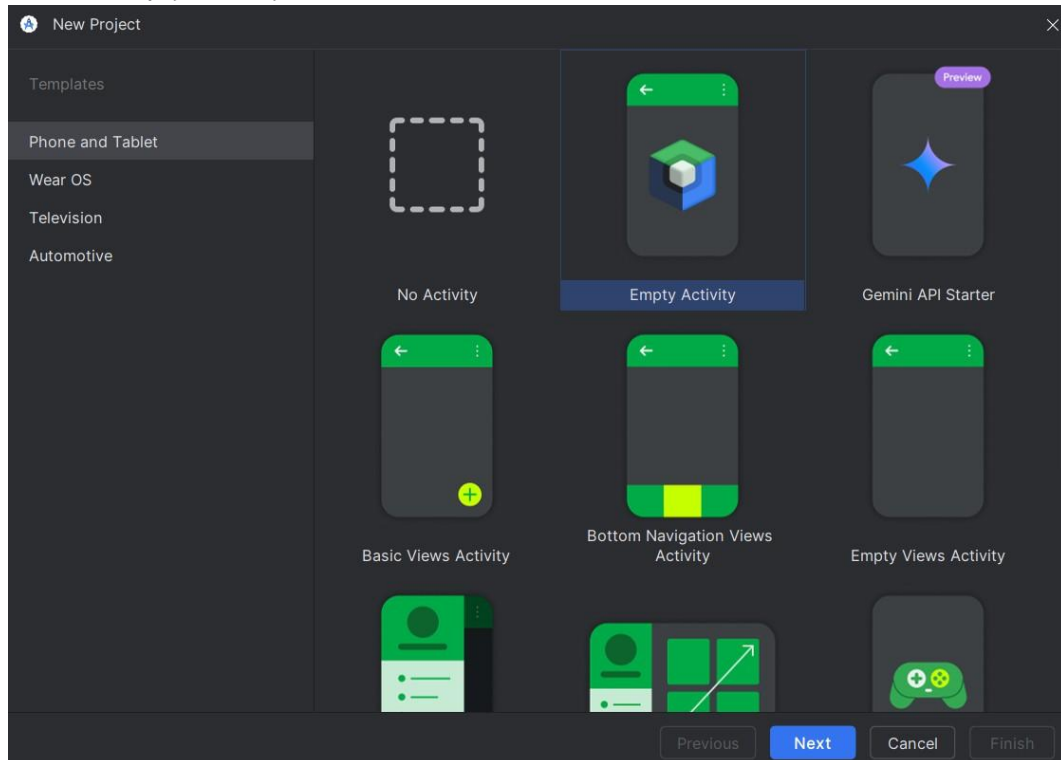


## UI AND LAYOUTS - I

### ANDROID APPLICATION DEVELOPMENT - 20CYS404

1. Open Android Studio and Create a new Project
2. Click on Empty Activity



3. Write a name for your Application and then click Finish.
4. In your MainActivity.kt file,

Add/Edit the following code:

i) MainActivity Class

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            ActivityAndLayout1Theme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    SelectionScreen(  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```

Explanation:

- ComponentActivity: A basic Android screen that supports Jetpack Compose.
- Bundle?: Stores saved data if the app was closed and reopened.
- enableEdgeToEdge() makes the content extend behind system bars
- setContent defines the UI using Compose
- Scaffold provides a basic material design layout structure
- SelectionScreen is our main composable that contains all UI elements

## ii) SelectionScreen Composable

```
// Main Screen Composition

@Composable
fun SelectionScreen(modifier: Modifier = Modifier) {
    // State hoisted to the screen level
    var selectedOption by remember { mutableStateOf("Select a Movie") }

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Option dropdown component
        OptionDropdown(
            selectedOption = selectedOption,
            onOptionSelected = { selectedOption = it }
        )

        Spacer(modifier = Modifier.height(32.dp))

        // Action button component
        SelectionActionButton(
            selectedOption = selectedOption
        )
    }
}
```

Explanation:

- selectedOption is a state variable that holds the currently selected option
- Column arranges its children vertically
- OptionDropdown is a custom dropdown component
- Spacer adds vertical space between components
- SelectionActionButton is a custom button component

### iii) OptionDropdown Composable

```
// Reusable Components

@Composable
fun OptionDropdown(
    selectedOption: String,
    onOptionSelected: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    var expanded by remember { mutableStateOf(false) }
    val options = listOf("F1", "Jurassic World Rebirth", "Superman", "Fantastic Four: First Steps")

    Box(modifier = modifier.fillMaxWidth()) {
        // Dropdown trigger
        OutlinedButton(
            onClick = { expanded = true },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(
                text = selectedOption,
                modifier = Modifier.weight(1f)
            )
            Icon(Icons.Default.ArrowDropDown, null)
        }
    }
}
```

```
// Dropdown menu

DropdownMenu(
    expanded = expanded,
    onDismissRequest = { expanded = false },
    modifier = Modifier.fillMaxWidth()
) {
    options.forEach { option ->
        DropdownMenuItem(
            text = { Text(option) },
            onClick = {
                onOptionSelected(option)
                expanded = false
                showToast(context, "Selected: $option")
            }
        )
    }
}
}
```

Explanation:

- expanded state controls whether the dropdown is visible
- options list contains the available choices
- OutlinedButton triggers the dropdown when clicked

- DropdownMenu shows the options when expanded is true
- Each DropdownMenuItem represents one selectable option
- When an option is clicked, it updates the selection and shows a toast

#### iv) SelectionActionButton Composable

```
@Composable
fun SelectionActionButton(
    selectedOption: String,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current

    Button(
        onClick = {
            showToast(context, "Button clicked! Selection: $selectedOption")
        },
        modifier = modifier.fillMaxWidth(),
        colors = ButtonDefaults.buttonColors(
            containerColor = MaterialTheme.colorScheme.primary,
            contentColor = MaterialTheme.colorScheme.onPrimary
        )
    ) {
        Text("Click Me")
    }
}
```

#### Explanation:

- Displays a button that shows a toast with the current selection when clicked
- Uses theme colors for consistent styling

#### v) Utility Function

```
//Utility Function

private fun showToast(context: Context, message: String) {
    Toast.makeText(
        context,
        message,
        Toast.LENGTH_SHORT
    ).show()
}
```

Explanation : Helper function to display short toast messages

## vi) Preview Composables

```
//Preview Composables

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun SelectionScreenPreview() {
    ActivityandLayout1Theme {
        SelectionScreen()
    }
}

@Preview
@Composable
fun OptionDropdownPreview() {
    ActivityandLayout1Theme {
        OptionDropdown(
            selectedOption = "F1",
            onOptionSelected = {}
        )
    }
}

@Preview
@Composable
fun SelectionButtonPreview() {
    ActivityandLayout1Theme {
        SelectionActionButton(selectedOption = "Superman")
    }
}
```

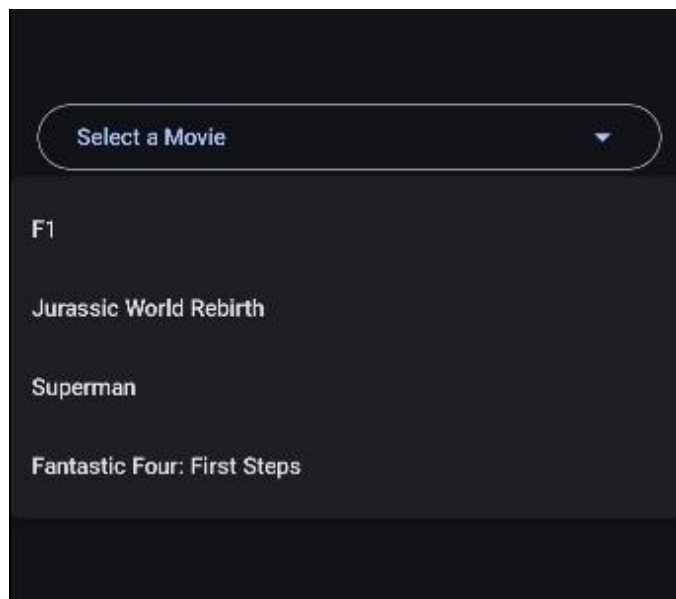
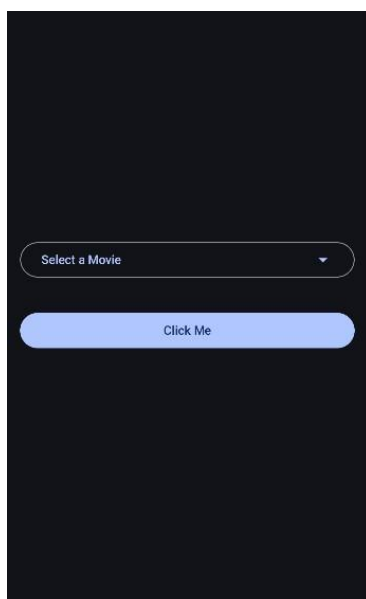
Explanation :

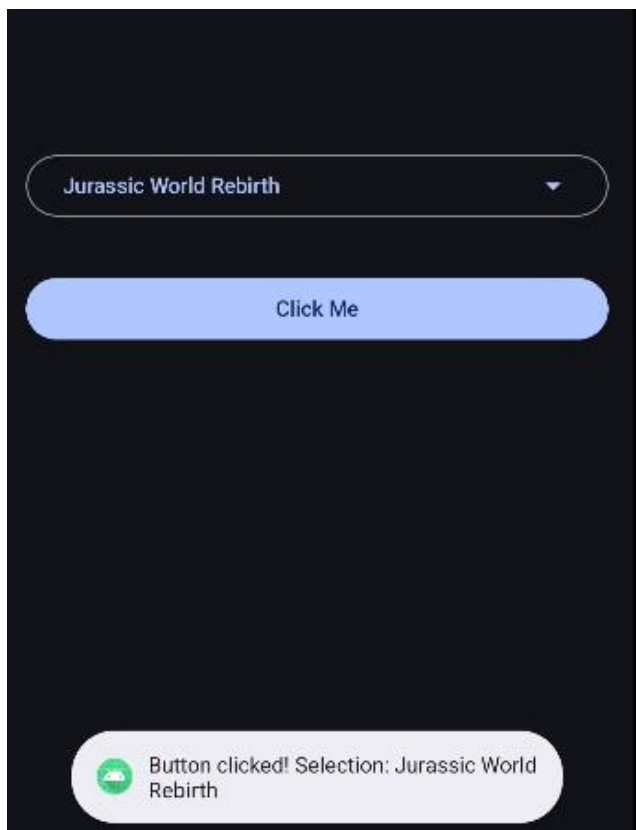
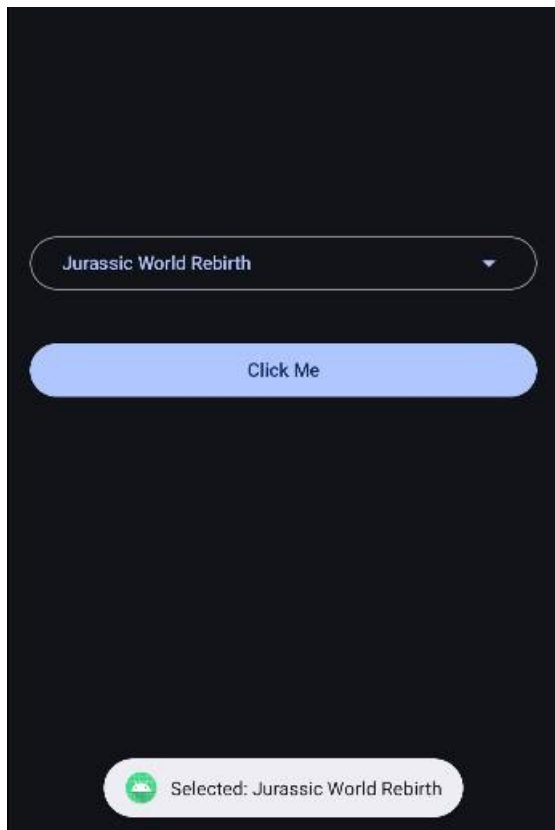
- Preview functions for Android Studio's design view
- Show how components look with different states

vii) Ensure the following the packages are imported:

```
import android.os.Bundle
import android.content.Context
import android.widget.Toast
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowDropDown
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateOf
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.learningtime.activityandlayout1.ui.theme.ActivityandLayout1Theme
```

viii) The desired output should look like this:





## Key Concepts:

1. State Management: Using `mutableStateOf` and `remember` to manage UI state.
2. State refers to the data stored about a component at a given point in time. It could be as simple as whether a toggle button is on or off, the text inside a text box, or more complex like the list of items retrieved from a database. When the state of a component changes, the UI is updated to reflect this new state.
3. Composition: Breaking UI into smaller, reusable components
4. Recomposition is the process where the UI automatically updates in response to state changes. When the state of an application changes, the parts of the UI that depend on that state automatically recompose, meaning they redraw themselves to reflect the new state.
5. When a composable function recomposes, the values inside it might get recalculated. Using **`remember`** helps in preserving the state or calculation, ensuring that it doesn't get recalculated unnecessarily, thus improving performance and maintaining consistency.
6. `mutableStateOf` is a function in Kotlin, specifically used in Jetpack Compose, Android's modern UI toolkit. It is used to create a mutable state variable. A mutable state variable is a variable whose value can change over time, and when it changes, it automatically recomposes or updates the parts of the UI that use this variable. This makes `mutableStateOf` essential for creating interactive and dynamic UIs in Android applications.
7. Material Design: Using Material 3 components for consistent styling
8. Layout: Arranging components with `Column`, `Spacer`, and modifiers
9. Interactivity: Handling user clicks and selections